IoT Discovery Protocols: a MQTT-D and UPnP Comparison in Dense Networks

Daniele Foschi
Alma Mater Studiorum Università di Bologna UNIBO
Bologna, Italy
daniele.foschi2@studio.unibo.it
https://github.com/DaniDF/MQTT-Discovery-vs-UPnP

***Abstract*—This dissertation provides a comparative evaluation of Universal Plug and Play (UPnP) and Message Queuing Telemetry Transport-Discovery (MQTT-D) based on two fundamental performance metrics: latency and robustness. The analysis examines the behaviour of both protocols under similar deployment conditions, with particular attention to dense network environments, where numerous devices simultaneously attempt to discover one another. The experimental findings indicate that UPnP consistently surpasses MQTT-D in both aspects, demonstrating lower latency and higher robustness across all tests. These results suggest that UPnP may represent a more effective solution for dense application scenarios in which rapid response times are essential.**

***Index Terms*—discovery, mqtt-d, upnp.**

I. Introduction

THE concept of Internet of Things (IoT) has been described through a variety of definitions in the literature. The International Telecommunication Union is *"A global infrastructure for the information society, enabling advanced services by interconnecting (physical and virtual) things based on existing and evolving interoperable information and communication technologies"* [1]. In a complementary manner, IEEE identifies a set of essential characteristics that distinguish IoT. Specifically, it is viewed as a networked infrastructure in which objects are uniquely identifiable, communicate through the Internet in at least part of their communication process, and are capable of sensing, actuating, or both. Furthermore, such objects are expected to be programmable through the integration of intelligence, while the services they provide should be accessible at any time, in any place, and for any entity.

In a environment populated by a large variety of different devices with different needs and capabilities designing a standard useful for communicating with them is a challenge. Discovery protocols are designed to reconcile the dichotomy between the requirement for highly specific, ad-hoc protocols capable of granular device control and the necessity for generalised frameworks that can accommodate a highly heterogeneous ecosystem of devices. By employing these protocols, an IoT device achieves autonomous configuration through active participation in the advertisement and solicitation of network-based services. Devices functioning as service providers systematically broadcast their available resources, the specific services they offer, and the operational guidelines for invocation, such as the appropriate Application Programming Interfaces (APIs). Furthermore, discovery protocols streamline the connectivity paradigm between clients and target resources or services. Consequently, this architecture ensures that clients are adequately equipped to transmit necessary input parameters to the services and successfully retrieve the corresponding computational results.

In particular this dissertation is focused on Universal Plug and Play (UPnP) and a protocol based on Message Queuing Telemetry Transport (MQTT) knwon as MQTT-Discovery (MQTT-D). UPnP is a discovery protocol without infrastructural nodes where each node can advertise and listen for services. In particular nodes that make available services are called *devices* and those searching for services are called *control-points*. In MQTT-D there is a third entity called *broker* that facilitates the communication between services and clients.

This dissertation wants to highlight the performance differences between UPnP and MQTT-D in IoT environment. The main focus is measuring and comparing latency and robustness in different deployment configurations: how different amount of data producers and consumers affect the protocol performances. Section II provides an overview of IoT protocol categorisation, followed by a detailed description of the two protocols under consideration. Section III outlines the requirements for the experimental evaluation and describes the methodology adopted for conducting the tests. Section IV presents and discusses the obtained results, while Section V summarises the main findings and concludes this dissertation.

II. Discovery Protocols for IoT

This section discuss about IoT discovery protocols with particular focus on MQTT-D and UPnP.

IoT environments are characterise by a large variety of different devices therefore there must be mechanisms available for automatic discovery of resources, their properties and capabilities as well as the means to access them [2]. In [3] are presented different categorizations for IoT discovery technologies, this dissertation will be focused only on *"Searching on My Network"*. A further distinction can be made based on the topology that these protocols implement, in particular they can be categorised into: centralised and distributed. In

centralised approaches, resources are registered in a directory that serves as intermediary between providers and users. In distributed approaches, users discover active resources through direct communication with resources in a Peer-to-Peer (P2P) fashion with no need for an intermediary directory [4].

UPnP is a P2P distributed discovery service without hierarchical structuring. In UPnP two entities can be distinguished: *control-point* the entity that look for services and resources and *device* the entity that make services available. A *device* can dynamically join a network and obtain a locally valid *IP* address. After it can make its capabilities visible and discover others, if present, and dynamically leave the network. UPnP uses *XML* for the description of the resources, services and action invocations.

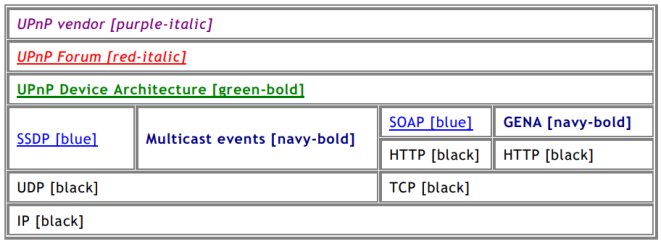

Fig. 1. UPnP Protocol Stack [5]

The UPnP protocol stack is based on four parts: Simple Service Discovery Protocol (SSDP) [5], HyperText Transfer Protocol (HTTP) [6], Simple Object Access Protocol (SOAP) [5] and General Event Notification Architecture (GENA) [5]. SSDP is a network protocol based on the Internet protocol suite for advertisement and discovery of network services and presence information. It is a text-based protocol based on a non fully compliant version of HTTP over UDP [5].

In SSDP, whenever a node intends to discover a *device*, it issues an M-SEARCH request specifying the desired device characteristics. Upon receiving this request, a *device* determines whether it satisfies the specified criteria and, if so, returns an appropriate response. Since the number of potential responses cannot be known in advance, a burst of simultaneous replies on the network may lead to overloading of the *control point*. To mitigate this risk, the *control point* includes the MX parameter in the search request [5]. This parameter defines the maximum number of seconds within which responding *devices* may transmit their replies. Consequently, each *device* waits for a random interval between 0 and the value specified by MX before sending its response.

SOAP is a messaging protocol specification for exchanging structured information in XML message format. It is used by the *control-point* for: (i) requesting the execution of services, Remote Procedure Call (RPC), with specified arguments, (ii) on the *device* and by the *device* for returning the results. GENA is an event notification protocol used by *devices* to publish variables updates to subscribed *control-points*. GENA supports both unicast and multicast event notification.

MQTT-D is a centralised discovery protocol MQTT-based [7]. MQTT is a lightweight publish-subscribe messaging protocol designed for constrained devices. MQTT-D is a metadata-exchange convention built on top of the MQTT protocol that standardises how devices and services describe themselves and publish their configuration to an MQTT broker. It enables automatic detection, registration, and integration of devices in home-automation or IoT ecosystems without requiring manual configuration.

The MQTT protocol assigns a specific Quality Of Service (QOS) level to each message, establishing the delivery guarantees required across potentially unreliable networks. The protocol defines three hierarchical levels of service: QOS 0 (At-Most-Once), which provides best-effort delivery without retransmission mechanisms; QOS 1 (At-Least-Once), which ensures delivery but permits the possibility of duplicate messages; and QOS 2 (Exactly-Once), the most complex level, which employs a multi-step handshake to guarantee that a message is delivered once and only once. Furthermore, publishers can append a *retained* flag to a message. This instructs the MQTT broker to persist the last message for a given topic, ensuring that any newly subscribed clients immediately receive the most recent data upon connection, independent of the initial publication event.

As MQTT-D does not define a standardised role-based classification for clients, this dissertation adopts, for the sake of terminological clarity, the same naming convention used in UPnP. Accordingly, entities that provide services are referred to as *devices*, whereas entities that discover and make use of such services are referred to as *control-points*.

	UPnP	**MQTT-D**
Architecture	Decentralized (P2P)	Centralized (Client-Broker)
Transport Protocol	UDP Multicast (HTTP over UDP)	TCP/IP
Network Scope	Local (LAN, Time-to-Live = 2 [5])	Global (not limited)
Data Format	XML	JSON

III. IoT Discovery Protocols Analysis

This dissertation aims to evaluate the performance of two of the main IoT discovery protocols, MQTT-D and UPnP. Such an evaluation is particularly relevant in dense IoT scenarios, including smart factory floors, smart cities, and extensive agricultural sensor grids, where system reliability and rapid device discovery are critical to maintaining overall network performance. For this reason the evaluated metrics for this study are latency and robustness. The first wants to measure the time needed for the execution of the first interaction between *control-point* and *device* after the discovery phase. The second wants to measure the ratio between the discovered devices and the actual deployed devices.

A. Implementation

To establish a testing environment, a fully compatible UPnP *device* and *control point* were implemented in accordance with

the official specification [5]. All major protocol components were developed, including SSDP, SOAP, and GENA [8]. For HTTP, the standard library provided by Go was employed. To further validate standards compliance and interoperability, a third-party UPnP client library was also used [9].

For MQTT-D, an existing implementation of the MQTT protocol was adopted, while only a wrapper for the MQTT-D data format was developed. As the MQTT broker, Mosquitto was selected due to its widespread adoption and popularity in IoT deployments.

MQTT-D defines two mechanisms for *device* presence discovery. The first relies on a retained message published by a *device* when it joins the network; this message is subsequently delivered by the broker to a *control point* when the latter subscribes to the corresponding topic [10]. The second mechanism is based on solicitation: the *control-point* publishes a message on a predetermined topic to which all *devices* are subscribed. Upon receiving such a solicitation message, each *device* responds by sending its discovery message [10].

The software developed for this work [8] supports both mechanisms; however, the experiments were conducted exclusively using the solicitation-based approach.

All software components developed by the author [8] were implemented in GoLang in order to balance high execution efficiency with the advantages of a higher-level programming language.

On top of each protocol module, a dedicated controller was implemented to expose ready-to-use APIss for the discovery of and subscription to *devices*. These controllers provide UPnP and MQTT-D objects that encapsulate the underlying control logic. In particular, RPC invocation, result processing, and error handling are managed internally, thereby offering a simplified abstraction that can be used without requiring knowledge of the internal operation of the underlying protocols.

To evaluate the protocols, a testing procedure was implemented to measure the time elapsed during each execution phase. All collected information was logged to the standard output.

The experimental environment consisted of two nodes: one hosting the *devices* and the other hosting the *control-points*. In the case of MQTT-D, a third node was additionally deployed to run the MQTT broker.

For each *device* and *control-point* configuration, a Bash script was developed to automate the testing process. The script was executed on the *control-points* node and, through SSH, instructed the *device* node to instantiate the required number of *devices*. At the end of each test, the *control-points* node ensured that no residual *device* instances were still running on the remote node.

The log output generated for each configuration was redirected and stored in a dedicated file. These log files were subsequently processed through a Python notebook to analyse the metrics and compute the statistics presented in this dissertation.

IV. Experimental Results

This section evaluates latency and robustness in a two-node UPnP deployment, in which one node runs the *devices* and the other runs the *control-points*. The MQTT-D deployment requires the addition of a third node, which hosts the MQTT broker. Each node is equipped with 12 core CPU, 16GB of memory and 2.5Gb lan interface.

Experimental evaluations were executed in a closed, dedicated network infrastructure to isolate the testing equipment. All experiments were performed over both wired and wireless (IEEE 802.11n [11]) connections. The deployment of *devices* and *control-points* was scaled incrementally: initially in unit steps from 1 to 10, and subsequently in decadal steps up to 100. For MQTT, each configuration was evaluated across QOS levels 0, 1, and 2. To mitigate statistical variance, each experimental combination was repeated 30 times, and the results were averaged.

The experiments tried to evaluate the effectiveness of the MX parameter in the UPnP protocol by removing the random wait time from the *device* response. The following subsections present the measured metrics for configurations of *devices* and *control-points* of value 1, 5, 10, 50, 100 but a more detailed version can be found in appendix A.

A. Latency comparison

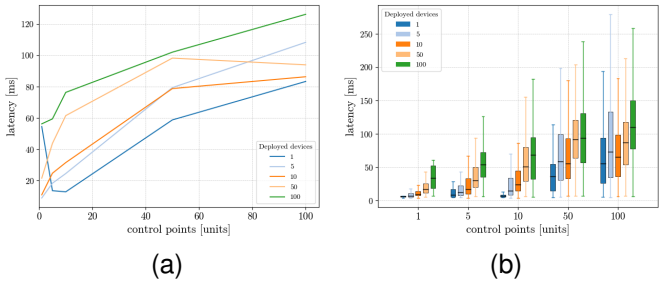

Fig. 2. (a) UPnP latency comparison with different *devices* and *control-points*. See 10. (b) Latency variance.

Figure 2 presents a comparison of UPnP latency across varying numbers of *devices* and *control-points*. Overall, the results show an approximately linear increase in latency as both parameters grow. The only deviation from this trend occurs in the configuration with a single *device*, which exhibits a latency decrease specifically within the range of one to five *control-points*. Beyond this interval, the latency follows the same increasing trend observed in the other configurations.

Figure 3 compares the latency of MQTT-D under varying numbers of *devices*, *control-points*, and QOS levels. The measurements reveal no substantial latency differences across the QOS configurations, aside from a slight increase observed for QOS 2 relative to QOS 0 and 1 when 100 *devices* are deployed. Latency increases approximately linearly as the number of control points grows from 1 to 50 in scenarios with fewer deployed devices. In contrast, for configurations involving a larger number of devices, the latency trend becomes noticeably flatter.

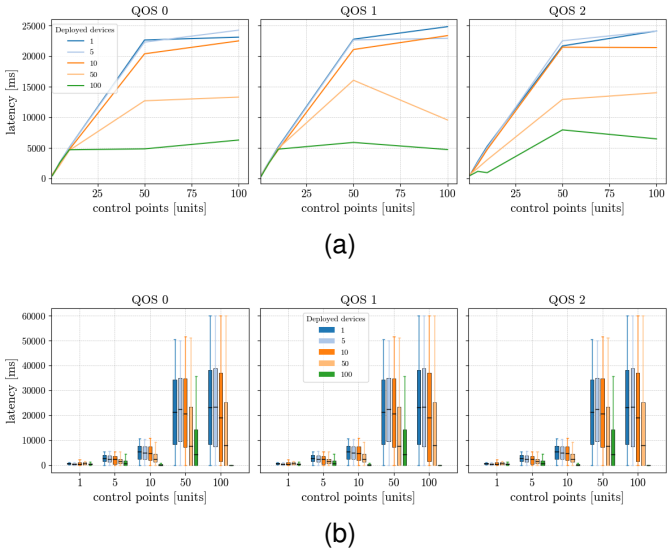

Fig. 3. (a) MQTT-D latency comparison with different *devices*, *control-points* and QOS. (b) Latency variance. See 12a.

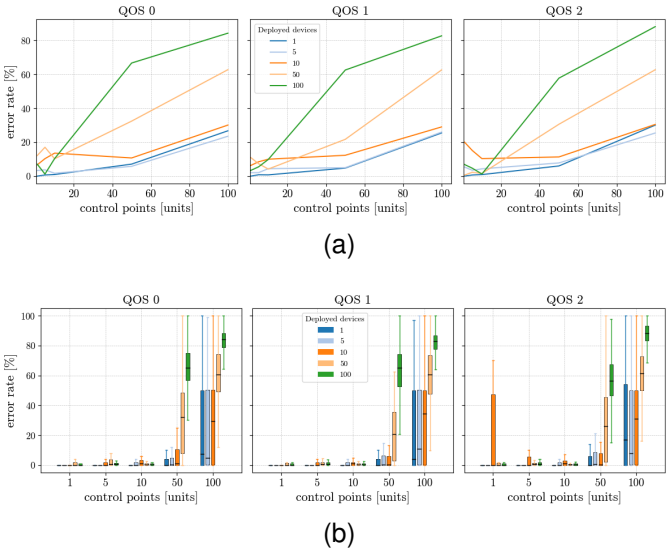

Fig. 5. (a) MQTT-D robustness comparison with different *devices*, *control-points* and QOS. (b) Robustness variance. See 13a.

1) Results comparison: The comparison between Figure 2 and Figure 3 shows that UPnP consistently achieves lower latency than MQTT-D. Across all tested configurations, UPnP remains below 130 ms, whereas MQTT-D reaches latency values of up to 25 s. Moreover, the latency of MQTT-D increases at approximately twice the rate observed for UPnP. Both protocols exhibit a noticeable reduction in the rate of latency growth once more than 50 *control-points* are deployed.

B. Robustness comparison

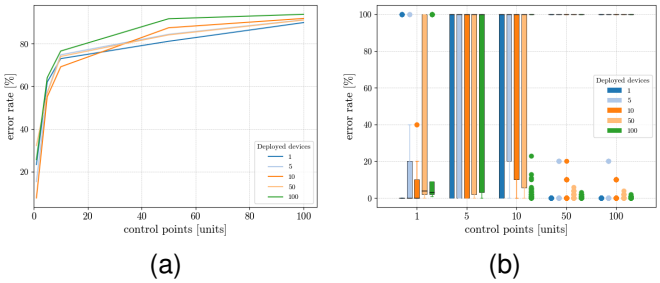

Fig. 4. (a) UPnP robustness comparison with different *devices* and *control-points*. (b) Robustness variance. See 11.

In figure 4 is shown a comparison of the error rate in UPnP with different *devices* and *control-points*. For the ratio between the number of *devices* discovered by the *control points* and the actual number of deployed *devices* is observed to be consistent across the different configurations (Figure 4). The error rate increases rapidly up to 10 deployed *control points* then it grows slowly towards 90%.

The Figure 5 shows a error rate comparison with different *devies*, *control-points* and QOS in MQTT-D.

The experimental results demonstrate a consistent behaviour across the evaluated QOS levels. For QOS 0, the error rate for configurations with 1 or 5 deployed *control-points* increases from 0% to 10%, reaching 10% at 50 *control-points*. In the case of 10 deployed *devices*, the error rate remains approximately constant up to 50 *control-points*, after which it increases at a rate comparable to that observed for 1 and 5 deployed *devices*. For configurations employing between 1 and 10 *control-points*, the error rate remains below 20%. Beyond this range, the error rate increases more rapidly as the number of deployed *devices* grows. In particular, the configuration with 100 deployed *devices* exhibits the steepest increase. Furthermore, all configurations show an approximately constant error-rate growth trend up to 50 *control-points*; beyond this threshold, the growth rate changes, becoming faster for configurations with fewer deployed *devices* and slower for those with a larger number of deployed *devices*.

1) Comparing the results: A comparison of the robustness of the two discovery protocols shows that MQTT-D demonstrates superior resilience. Specifically, MQTT-D outperforms UPnP in scenarios with lower numbers of deployed *devices*, achieving a maximum error rate of approximately 30%, in contrast to the 90% observed for UPnP. In configurations with higher device deployments, MQTT-D continues to perform more reliably than UPnP; in particular, its error rate increases at an almost linear pace, whereas UPnP exhibits a sharp rise, reaching 80% almost immediately.

C. UPnP MX = 0

Figure 6a shows the latency of UPnP when all the artificial waiting times are removed.

A comparison between Figure 6a and Figure 2 reveals an inverse latency trend. In standard UPnP, deployments with fewer *devices* are associated with lower latency. By contrast, when the artificial waiting times are removed, configurations with a higher number of deployed *devices* exhibit the lowest latency. This effect is particularly evident in scenarios involving a larger number of *control-points*. Overall, the two experiments produce broadly similar results, although standard UPnP still achieves slightly better performance.

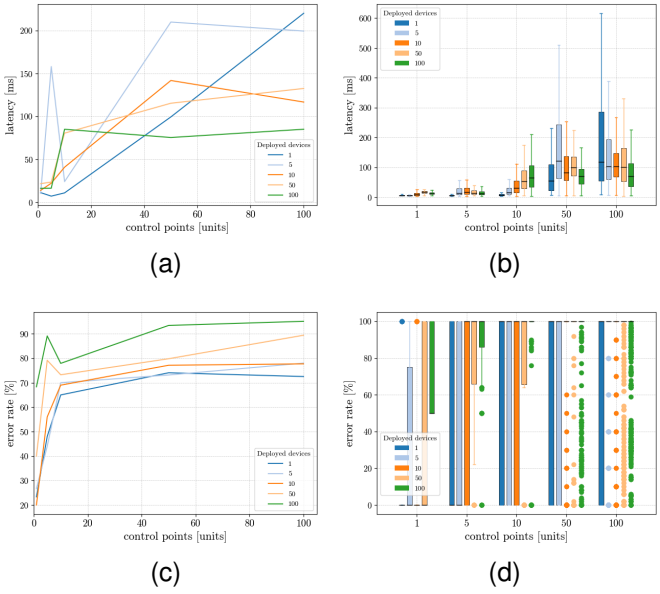

Fig. 6. UPnP (a) latency and (c) robustness different *devices control-points*. (b) quartile latency variance. (d) Robustness variance.

In terms of robustness, Figure 4, when compared with Figure 6c, exhibits a more compact distribution of results. In particular, all deployed *device* configurations show more homogeneous behaviour with respect to those reported in Figure 6c. According to this metric, the two configurations exhibit only negligible differences.

D. GENA

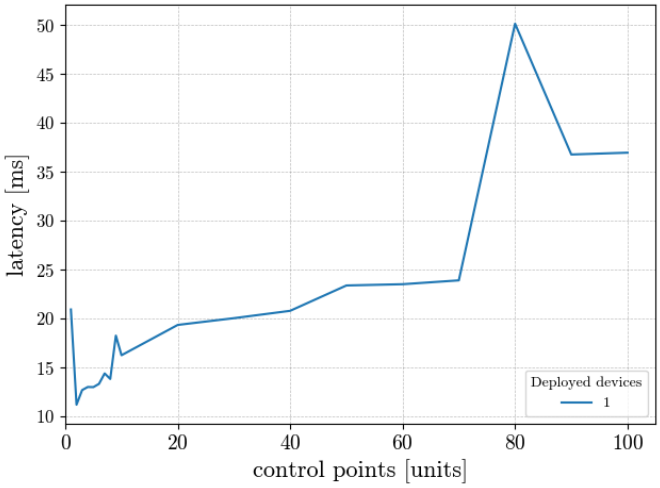

Fig. 7. GENA latency with different *control-points*.

The Figure 7 shows the delivery latency of unicast events as a function of the number of subscribed *control-points*. Data presented in Figure 7 indicates that for configurations with fewer than 2 subscribers, latency and the number of *control-points* share an inverse relationship. Conversely, exceeding this limit results in a direct correlation, where additional *control-points* lead to increased latency.

E. Wireless vs Wired

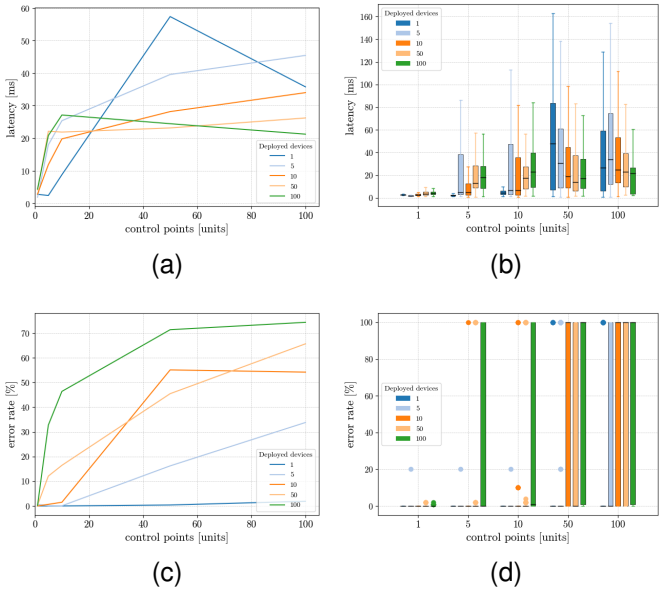

Fig. 8. Wired UPnP (a) latency and (c) robustness different *devices control-points*. (b) Latency variance. (d) Robustness variance. See 10.

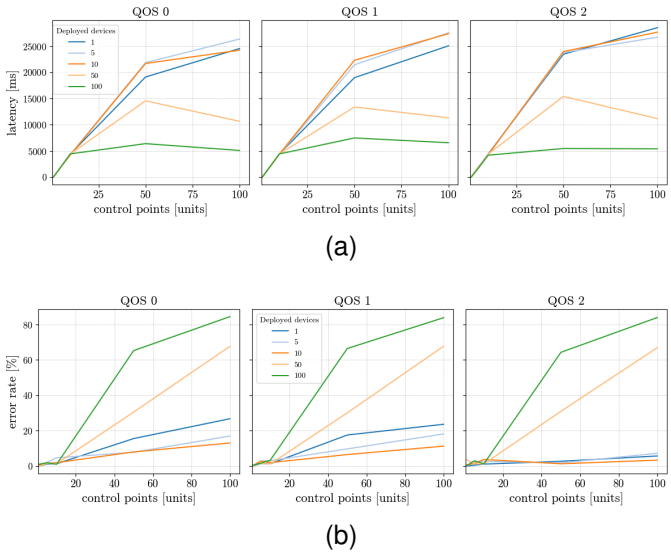

Fig. 9. Wired MQTT-D (a) latency and (b) robustness different *devices control-points*.

Figure 8 reports the wired latency and error-rate characteristics observed across various *device* and *control-point* configurations in UPnP. The results for UPnP indicate a clear benefit from the wired connection, with latency values approximately half of those measured in the wireless experiments. Notably, in the wired setup, deployments with fewer *devices* are associated with higher latency (Figure 8a). With respect to robustness, the single-*device* deployment maintains an error rate close to 0% across all tested configurations. A comparison between wired and wireless scenarios further shows that, in the former, the error rate increases more linearly, whereas in the latter it rises almost immediately to 80% and then further increases to

90%. An additional observation concerns the reduced latency measured in the 100-*devices* configuration. This behaviour can be explained by the corresponding error rate, which exceeds 70%, indicating that most of the deployed *devices* are not successfully discovered. Therefore, the lower latency observed in configurations with a higher number of deployed *devices* is a consequence of the limited number of *devices* actually discovered.

Figure 9 presents the wired latency and error-rate results across different *device*, *control-point*, and QOS configurations. The MQTT-D experiments exhibit consistent behaviour across all QOS configurations. In terms of latency, a slight increase is observed as the QOS level increases. Configurations with a higher number of deployed *devices* achieve the lowest latency, but also exhibit the highest error rate. A modest yet significant improvement in robustness can be observed with increasing QOS, particularly in scenarios with a smaller number of deployed *devices*.

F. Discussion

The experiments show that UPnP achieves significantly lower latency than MQTT-D. For configurations ranging from 1 to 50 deployed *control points*, UPnP records latency values between 60 and 100 ms, whereas MQTT-D exhibits latencies ranging from 5 to 20 s. This substantial difference can be attributed to the lighter protocol overhead of UPnP, which relies primarily on UDP, in contrast to MQTT-D, which is based on TCP.

Since in UPnP the RPC mechanism is implemented through SOAP that has a client-server approach, it is also interesting a comparison between GENA and MQTT-D both publisher-subscriber.

In the single-*device* deployment scenario, GENA exhibits the lowest latency among all QOS configurations evaluated for MQTT-D. More generally, the results show that GENA achieves lower delivery times than MQTT-D when only one *device* is deployed. Because all experiments involving GENA were conducted in unicast event-delivery mode, the observed performance advantage of GENA cannot be attributed to multicast communication.

A comparison of Figure 2 with Figure 8a, indicates that UPnP over a wired connection achieves significantly lower latency, with measured values reduced by nearly half. Moreover, a comparison of Figure 4 with Figure 8c shows that the wired configuration also improves robustness. In particular, the single-*device* deployment exhibits an error rate close to 0%, and each subsequent increase in deployed *devices* leads to a gradual degradation in performance across the different *control-point* deployments. This behaviour contrasts with that observed in the wireless scenario, in which all deployed *devices* exhibit a more uniform trend across configurations as the number of *control points* increases.

A comparison between the wireless and wired MQTT-D configurations does not reveal any significant improvement in either latency or robustness. The experimental results are nearly identical for both metrics across the two scenarios.

As discussed previously, the latency measurements exhibit a tendency to increase at a lower rate once the number of deployed *control-points* exceeds 50. This behaviour is more pronounced in MQTT-D than in UPnP and can be explained by comparing Figure 3 with Figure 5. In particular, when the number of deployed *control-points* reaches 100, the number of valid connections does not significantly exceed that observed with only 50 *control-points*. This indicates that, from 50 to 100 *control-points*, the total number of successful connections remains nearly constant, while their distribution over time becomes more concurrent.

As an illustrative example, consider MQTT-D with QOS 0 and 100 deployed *devices*. When 50 *control-points* are deployed, the expected number of connections is 5000; however, given an error rate of approximately 67%, the number of successful connections is reduced to about 1650. When the number of deployed *control-points* is increased to 100, the measured error rate rises to 83%, yielding approximately 1700 successful connections. Although this represents a slight increase in the number of valid connections, it occurs under a substantially higher level of concurrency, with 100 *devices* and 100 *control-points* interacting simultaneously.

For this reason, latency growth nearly saturates once 50 *control-points* are deployed. The additional *control-points* do not generate a significant number of new successful connections with the *devices*, and therefore produce only a marginal impact on the measured latency.

Based on the evaluations conducted in this dissertation, UPnP achieves the lowest latency, whereas MQTT-D provides the highest level of robustness. These results suggest that UPnP is more suitable for IoT networks with a limited number of *devices*, such as smart homes or small-scale facilities. In contrast, MQTT-D, due to its greater robustness, appears more appropriate for dense IoT environments, in which a large number of *devices* must perform discovery and exchange data concurrently.

V. Conclusions

This dissertation undertook a comparative evaluation of MQTT-D and UPnP in dense network environments, with particular focus on latency and robustness under varying deployment configurations of *devices* and *control-points*. The experimental scenarios were assessed within a real network infrastructure specifically established for the purposes of this study. Each configuration was tested in both wired and wireless environments, the latter employing IEEE 802.11n technology.

For the purposes of these experiments, a fully compliant UPnP *device* and *control-point* were implemented. Nevertheless, to ensure compliance with the specifications, part of the *control-point* functionality relied on a third-party library.

The experimental results show that UPnP achieves the lowest latency across all evaluated configurations, making it more suitable for small-scale IoT networks. In contrast, MQTT-D demonstrates greater robustness across all adopted QOS levels, which makes it more appropriate for denser IoT deployments.

A comparison of wired and wireless deployments shows that UPnP benefits significantly from the wired configuration, with lower latency and greater robustness, whereas performance in

the wireless case degrades more rapidly as the deployment scales. In contrast, MQTT-D exhibits nearly identical latency and robustness in both scenarios. Therefore, the connection type has a significant impact on UPnP but a negligible effect on MQTT-D.

Future research could investigate how UPnP and MQTT-D behave with real scenario data, investigating how payloads affect latency and robustness. In addition a further study could repeat this experiments in smart homes or industrial plant networks and evaluate how performances degrades in a shared network.

APPENDIX

A. UPnP

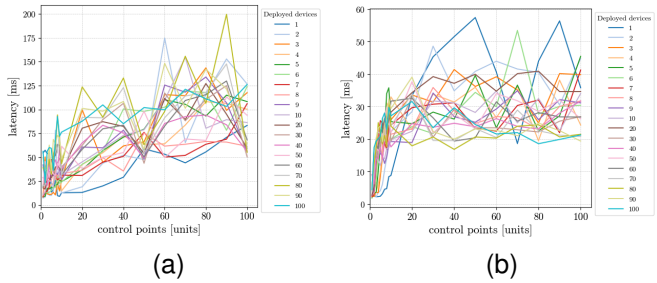

Fig. 10. UPnP (a) wireless and (b) wired latency

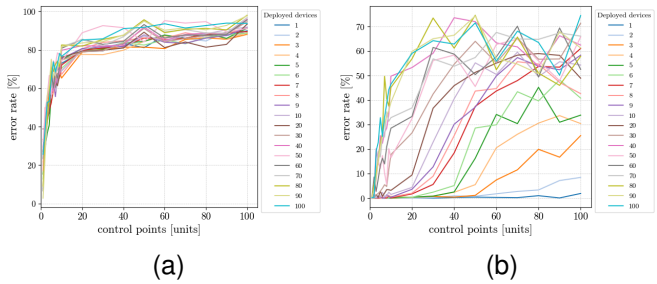

Fig. 11. UPnP (a) wireless and (b) wired robustness

B. MQTT

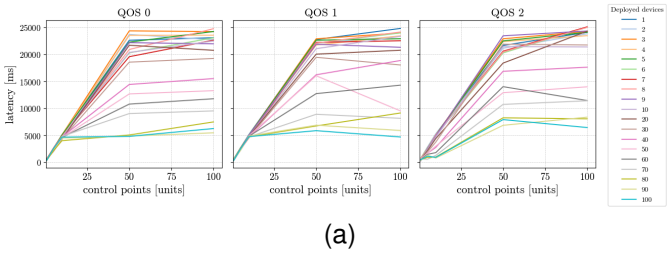

Fig. 12. MQTT latency

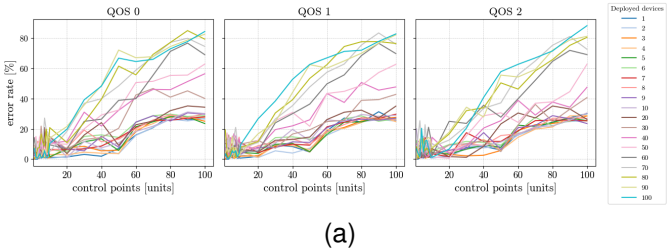

Fig. 13. MQTT robustness

Daniele Foschi Mater Degree student in Computer Engineering at University of Bologna.

REFERENCES

[1] I. T. Union, "Series y: Global information infrastructure, internet protocol aspects and next-generation networks," ITU-T, Recommendation Y.2060, February 2012. [Online]. Available: https://handle.itu.int/11.1002/1000/11559

[2] S. K. Datta, R. P. F. Da Costa, and C. Bonnet, "Resource discovery in internet of things: Current trends and future standardization aspects," in *2015 IEEE 2nd World Forum on Internet of Things (WF-IoT)*, 2015, pp. 542–547.

[3] A. Bröring, S. K. Datta, and C. Bonnet, "A categorization of discovery technologies for the internet of things," in *Proceedings of the 6th International Conference on the Internet of Things*, ser. IoT '16. New York, NY, USA: Association for Computing Machinery, 2016, p. 131–139. [Online]. Available: https://doi.org/10.1145/2991561.2991570

[4] K. Khalil, K. Elgazzar, and M. Bayoumi, "A comparative analysis on resource discovery protocols for the internet of things," in *2018 IEEE Global Communications Conference (GLOBECOM)*, 2018, pp. 1–7.

[5] O. C. Foundation, "Upnp device architecture 2.0," Open Connectivity Foundation, Tech. Rep., April 2020, document Revision Date: April 17, 2020. [Online]. Available: https://openconnectivity.org/upnp-specs/UPnP-arch-DeviceArchitecture-v2.0-20200417.pdf

[6] T. Berners-Lee, R. Cailliau, A. Luotonen, H. F. Nielsen, and A. Secret, "The world-wide web," *Commun. ACM*, vol. 37, no. 8, p. 76–82, Aug. 1994. [Online]. Available: https://doi.org/10.1145/179606.179671

[7] M. Achir, A. Abdelli, and L. Mokdad, "A taxonomy of service discovery approaches in iot," in *2020 8th International Conference on Wireless Networks and Mobile Communications (WINCOM)*, 2020, pp. 1–6.

[8] DaniDF, "MQTT-Discovery vs UPnP," https://github.com/DaniDF/MQTT-Discovery-vs-UPnP, 2026.

[9] Huin, "goupnp," https://github.com/huin/goupnp, 2026.

[10] O. H. Foundation, "Mqtt discovery," https://www.home-assistant.io/integrations/mqtt/#mqtt-discovery, 2026.

[11] I. C. Society, "Ieee standard for information technology– local and metropolitan area networks– specific requirements– part 11: Wireless lan medium access control (mac)and physical layer (phy) specifications amendment 5: Enhancements for higher throughput," *IEEE Std 802.11n-2009 (Amendment to IEEE Std 802.11-2007 as amended by IEEE Std 802.11k-2008, IEEE Std 802.11r-2008, IEEE Std 802.11y-2008, and IEEE Std 802.11w-2009)*, pp. 1–565, 2009.